Open Evidential Tool Bus (O-ETB) Development Version v1.1.1 (Prototype #1) User Manual v5 July 2024

Contents

Introduction 5

1. Version History 6
1.1 Overview of v1.1.0 6
1.1.1 Repository structure 6
1.1.2 Functional configurations 6
1.1.3 Known limitations of current version 7
1.2 Features foreseen in near-future versions 8
1.2.1 Assurance Server configuration 8
1.2.2 Import of MedSecurance Model information 8
1.2.3 Certification project registry 8
1.2.4 Development of MedSecurance argument patterns and workflows 8
1.2.5 Development of Tool Agents for MedSecurance tools 9
1.3 Summary of earlier work leading to O-ETB 9
1.3.1 CITADEL Project 9
1.3.2 D-MILS Project 9
1.3.3 ETB2 - fortiss 9
1.3.4 ETB Concept – SRI International 10

2. O-ETB Assurance Tool and Assurance Server Description 11
2.1 O-ETB repository organisation 11
2.1.1 Development directories 12
2.1.2 Database directories 12
2.1.3 Source Code 13
2.1.4 Knowledge Base 13
2.1.5 Assurance case and evidence repositories 13
2.1.6 Certification Assurance Package 14
2.2 'etb' Assurance Tool 14
2.2.1 Command line options 14
2.2.2 Assurance Tool interactive commands 14
2.2.3 Timing Commands 18
2.2.4 Command procedures and scripts 18
2.3 'etb-server' Assurance Server 18
2.3.1 Server assurance case sessions 18
2.3.2 Server certification projects 19
2.3.3 Assurance Server APIs 19
2.4 Assurance case patterns 22
2.4.1 Basic syntax of pattern language 23
2.4.2 Parameterized elements in the pattern language 23

3. Installation, Build and Operation 25
3.1 Overview 25
3.2 Prerequisites 25
3.2.1 Install prerequisite software 25
3.2.2 Install the O-ETB source files and/or executables 26
3.3 Building and Running the 'etb' assurance tool 26
3.3.1 Compile the 'etb' policy tool 26
3.3.2 Test the installed 'etb' tool 27
3.3.3 Running the examples 28
3.4 Building and Running the 'etb-server' 28
3.4.1 Install the O-ETB source files and/or etb-server executable 28
3.4.2 Initiating the 'etb-server' 29
3.4.3 Test the installed 'etb-server' 29
3.5 Extending Assurance Tool and Server 29
3.6 Assurance Tool & Server Deployment for MedSecurance 30
3.7 Application to Use Cases and Building the Knowledge Base 30

4. Principles of Operation 32
4.1 Architecture 32
4.2 O-ETB components 32
4.2.1 Functional components 32
4.2.2 Data structures and stores components 33
4.2.3 Packaging of core functions 33
4.3 Component interactions 33
4.4 Operating modes 35
4.5 Argument pattern instantiation 35
4.6 CASES repository 38
4.6.1 Assurance case directories 38
4.6.2 Persistent assurance case storage 38
4.6.3 Working assurance case cache 38
4.7 Assurance Evidence 39
4.7.1 Evidence Validation 39
4.7.2 Evidence artefacts 39
4.7.3 Evidence categories 40
4.7.4 Evidential tool agents 40
4.7.5 Evidence artefact validation status 42
4.7.6 Re-synchronisation 42
4.8 EVIDENCE repository 42
4.8.1 Evidence category directories 42
4.8.2 Persistent evidence storage 42
4.8.3 Working evidence cache 42
4.9 Certification Assurance Package (CAP) 42
4.9.1 Tailoring the CAP to certification requirements 43
4.9.2 Organising multiple certification projects 43
4.9.3 Text file artefacts 43
4.9.4 HTML directory artefacts 43

5. Assurance Tool and Server Examples 45
5.1 Example 1 – O-ETB Tool instantiating a simple pattern 45
5.1.1 Instantiating an argument pattern 45

5.1.2 Exporting assurance case as text *45*
5.2 Example 2 – Assurance Tool exporting an assurance case 45
5.2.1 Attaching an existing assurance case *45*
5.2.2 Exporting assurance case as HTML *46*
5.2.3 Browsing an exported assurance case *46*
5.3 Example 3 – Assurance Tool loading a model and instantiating a pattern list 46
5.4 Example 4 – MedSecurance RPMS example 46

6. References 48

7. Glossary of terms and abbreviations 49

8. Annex A: O-ETB Configuration Identification 50
8.1 File identification 50
8.2 Module identification 51

9. Annex B: Populating the O-ETB Knowledge Base 54
9.1 Model information 54
9.1.1 Architectural model *54*
9.1.2 Properties *55*
9.2 Argument Patterns 55
9.3 Workflow information 55
9.3.1 Tool flows *55*
9.3.2 Workflow programs *56*
9.4 Certification requirements 56
9.4.1 Content/process requirements *56*
9.4.2 Goal-based requirements *56*
9.4.3 Presentation requirements *56*
9.5 Evidence-related knowledge 56
9.5.1 Evidence categories and artefact kinds *56*
9.5.2 Evidential tools *58*
9.6 General knowledge 58

10. Annex C: O-ETB Testing 59
10.1 Functional Tests 59
10.2 Volume and Load Tests 59
10.3 Performance and endurance tests 59
10.4 HTTP API correctness tests 60

Figures

Figure 1 O-ETB repository top-level structure 12
Figure 2 Database structure 13
Figure 3 Functional architecture 32
Figure 4 Component interaction sequence 34
Figure 5 Tool agents integrate external tools 35
Figure 6 Major modes and actions 35
Figure 7 Pattern instantiation rewrite rules 36
Figure 8 Tailoring the CAP to certification requirements 43

Tables

Table 1 Assurance Tool general purpose commands 15
Table 2 Assurance Tool data and control commands 17
Table 3 Assurance Server APIs 19
Table 4 Example 1 – instantiating a simple pattern 45
Table 5 Example 2 – exporting an assurance case 45
Table 6 Example 2 – instantiating a pattern list from a model and exporting as HTML 46
Table 7 Terms and Abbreviations 49
Table 8 O-ETB v1.1.0 directory tree 50
Table 9 O-ETB v1.1.0 source code modules 51
Table 10 Evidence artefacts and categories 56
Table 11 Pattern instantiation correctness tests 60
Table 12 Assurance Server HTTP API correctness tests 61

INTRODUCTION

This User Manual version 4 for the Open Evidential Tool Bus (O-ETB) Version 1.1.0 is a work in progress. A few sections are still under-developed or undeveloped, but revisions and additions of material are actively ongoing. It is treated as a living document with Multiple versions of the User Manual may apply to the same version of O-ETB as identified in the title. The version number of the User Manual (currently v4) should be used to identify newer versions which supercede lower numbered versions.

1. VERSION HISTORY

This section presents a synopsis of versions of the Open Evidential Tool Bus (O-ETB) in reverse chronological order. The first subsection, which pertains to the current version, is followed by a preview of the anticipated next version. When a new version is released the preview subsection will be removed from the document and a new version subsection and new preview subsection will be added in the *front* of the then-current first subsection.

1.1 OVERVIEW OF V1.1.0

Version 1 of O-ETB is a continuing development derived from Adaptive MILS ETB v3 (AM-ETB), implemented by Université Grenoble Alpes as part of the CITADEL Project, in continuation of automated assurance case construction support implemented by University of York as part of the D-MILS Project. For more on this legacy work see Section 1.3.

O-ETB is intended to be an open source development that will be available on GitHub. O-ETB v1 and anticipated future adaptations and improvements are being developed under the auspices of the European Commission's MedSecurance Project. Please consult the MedSecurance design and specification documents (e.g. D2.2 Requirements & Design of Automated IoMT Security Analysis & Assurance Toolbox) for a requirements specification of the features foreseen for this development. Prototype #1 of O-ETB for the MedSecurance Project is designated as O-ETB v1.1.0.

1.1.1 Repository structure

O-ETB adopts a new repository structure, consolidating source code and adding new directory structure elements for a dynamic knowledge base, repositories for changeable persistent data comprising assurance cases and assurance evidence, artefact collections comprising certification assurance packages, build space, and documents. Further details of the repository structure may be found in Section 2.1.

1.1.2 Functional configurations

This version of O-ETB is configured as an interactive *Assurance Tool* though it contains some new groundwork for an *Assurance Server* configuration. Numerous source files are shared to build both the Assurance Tool and the Assurance Server.

Functionally, the Assurance Tool has been extended with a new extensible command line interpreter for enhanced development, testing, and user interaction; and initial steps toward the background-running server that will enable concurrent use by multiple users and processes in the construction of separate assurance cases and certification assurance packages.

More information about the functional configurations mentioned here may be found in the main body of this User Manual and its Annexes.

1.1.2.1 Assurance Tool configuration

The Assurance Tool has been extended with a command line interpreter, including commands that provide access to internal assurance case and evidence functions (also to be accessed via external API invocations), knowledge base loading and inspection, and functions to facilitate ongoing development such as interactive tracing and testing.

The Assurance Tool may be invoked to operate on the centralized repository or on a working repository that has the same structure. This mode of use is intended for development, testing, and/or independent assurance case and certification development efforts.

1.1.2.2 Assurance Server configuration

The source code for the current version includes extensions to initiate a server mode of background operation and the makefile contains a target for an Assurance Server configuration. However, the code does not yet include all minimal server functionality nor an implementation of the proposed APIs.

1.1.3 Known limitations of current version

O-ETB Version 1.1.0 has the following known limitations:

- Error handling is rudimentary at present. A log is constructed of some error messages. This should be extended to informational log messages. In the command interpreter command failures are reported but the failure conditions may not be clearly reported. Generally, error reporting and other informational messages need to be more thoroughly implemented and tested.

- Tool agents that enable the integration into the O-ETB workflow of evidence creation and validation tools and techniques are limited to few tools and simple examples. These need to be extended for additional evidence categories and the corresponding evidence-generating tools that are applied to contemporary development and assurance projects.

- The provided model example is simple and limited. The set of examples needs to be extended to include additional project-relevant and more complex examples.

- The library of assurance case argument patterns need to be extended with patterns that novel and diverse assurance methodologies and tools, e.g. Threat Vulnerability Risk Analysis (TVRA) and associated tools.

1.2 FEATURES FORESEEN IN NEAR-FUTURE VERSIONS

1.2.1 Assurance Server configuration

A distinct "etb-server" executable is built to provide a distinct entry point to the O-ETB assurance server mode of operation. The server will run in the background and be interacted with by other processes through a RESTful API in addition to the current filesystem-based interaction protocols. This API will provide programmatic access to some functions in common with those of the command language. The Assurance Server is also discussed in Sections 1.1.2.2, 1.2.1, and 2.3.

New RESTful HTTP APIs for the assurance server have begun to be identified as part of the activities to develop O-ETB Prototype #1 but are not yet been implemented. The Prototype #2 will implement the basic server and at least the minimum subset of these identified APIs. The APIs are also discussed in Section **Error! Reference source not found.**

1.2.2 Import of MedSecurance Model information

To be supplied. A preliminary study of the model information currently needed by O-ETB and how it might be provided by other MedSecurance modelling activities is provided in Annex A.

1.2.3 Certification project registry

The certification project registry will provide operations to create a named certification project and to keep certification process and artefact content and presentation requirements associated with the project. The certification requirements permit specific argument patterns, evidence categories, and workflows to be selected from the Knowledge Base for guidance of the assurance activities and assurance case development and ultimately the rendering of the required artefacts in the project's certification assurance package (CAP). Additional comments about the certification project registry may be found in Section 4.9.

1.2.4 Development of MedSecurance argument patterns and workflows

Assurance case argument patterns and workflows that embody the MedSecurance methodology will be developed in conjunction with the initial applications of the methodology and tools, which will at first be guided by the developers and use cases. The resulting patterns and workflows are effectively scripts that allow the process to be repeated automatically as the use cases evolve and as the patterns are applied to other system that will also employ the MedSecurance methodology, architectural patterns and tools.

More to be supplied.

1.2.5 Development of Tool Agents for MedSecurance tools

The automated tools chosen for the generation of evidential artefacts within the MedSecurance methodology will need to be integrated into the O-ETB workflow. This is accomplished by the development of small wrapper programs, referred to as ***tool agents***, that adapt individual external tools, including MedSecurance developed tools and chosen third-party tools, to the Assurance Tool and Assurance Server interaction protocols defined by O-ETB.

More to be supplied.

1.3 SUMMARY OF EARLIER WORK LEADING TO O-ETB

O-ETB represents an evolutionary development of the approach and implementation of assurance case automation support continuing the spirit and developments of the D-MILS (2012-2015) and CITADEL (2016-2019) EC projects. These previous assurance efforts were done in the context of the MILS architectural approach, encompassing platform, architecture and model-based development and verification methods for systems requiring high-assurance of security, safety and other key system properties. These efforts were supported by a methodology and tool set supporting rigorous specification and verification of MILS systems.

MedSecurance and other contemporary projects require a broadening of the scope of application of these methods and tools embodied in earlier work from the MILS platform and architecture environment to alternative and more generalized environments and a different and more diverse selection of assurance supporting tools and techniques.

The immediate predecessor of O-ETB was the CITADEL Project's Adaptive MILS ETB (AM-ETB). This work was in turn preceded by the D-MILS Project's assurance case pattern and M-BAC tool effort.

The O-ETB effort and its predecessor AM-ETB were inspired by, but are a separate branch from the original concept and implementations at SRI International and fortiss.

1.3.1 CITADEL Project

Description of assurance case patterns and the AM-ETB tool - To be supplied.

1.3.2 D-MILS Project

Description of assurance case patterns and the M-BAC tool - To be supplied.

1.3.3 ETB2 - fortiss

Description of a collaboration between SRI and fortiss for a "second" implementation of ETB - To be supplied.

1.3.4 ETB Concept – SRI International

Description of original concept by John Rushby and initial prototype implementation at SRI International - To be supplied.

2. O-ETB ASSURANCE TOOL AND ASSURANCE SERVER DESCRIPTION

The O-ETB Assurance Tool and Assurance Server represent two different modes of access to common underlying functionalities that provide for the construction of assurance cases, including their supporting evidence, and the rendering of these assurance cases with their supporting evidence in a form appropriate for independent evaluation.

The Assurance Tool provides a command-based interface that can be used interactively and/or in a batch mode to run command scripts. The Assurance Server provides programmatic access to the core functions through an API. Both may be run on a shared centralised file structure, or, for development, testing or experimentation, on independent instances of the same file structure, as determined by configuration options and invocation arguments. As part of that file structure, assurance cases and evidence are constructed incrementally and stored persistently in the CASES and EVIDENCE Repositories respectively.

The core workflow of O-ETB utilises *tool agents* to incorporate external evidence-generating tools into the workflow and to store their results in a canonical form into the EVIDENCE Repository.

Tool agents serve as adaptors to include external evidence-producing tools into the O-ETB core workflow. A tool agent embodies knowledge about what data is required for successful running of the tool, executing the tool, interpreting the results of executing the tool, and reporting the results back to the ETB core workflow through interaction protocols that include a combination of tool agent return codes, calls to the O-ETB Assurance Server through its RESTful Evidence Management Interface and by directly manipulating files in the EVIDENCE Repository. O-ETB includes rendering functions that use information from the repositories to create collections of artefacts for certification efforts according to requirements of a particular target certification domain.

2.1 O-ETB REPOSITORY ORGANISATION

The O-ETB repository structure consolidates source code and adds new directory structure elements for a dynamic knowledge base, repositories for changeable persistent data comprising assurance cases and assurance evidence, artefact collections comprising certification assurance packages, build space, and documents. Figure 1 shows the top-level structure of the O-ETB repository. This includes subdirectories related to both development, including src, doc, BUILD and TEST, and subdirectories the hold the runtime and persistent data structures, including KB, REPOSITORY and CAP. Annex A contains a complete list of files in the O-ETB repository.

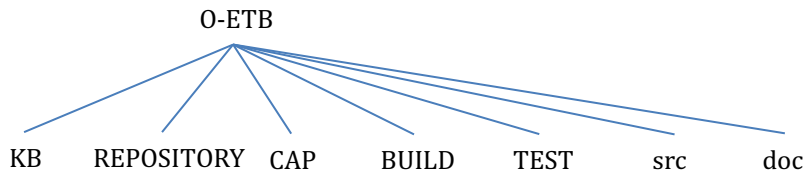

Figure 1 O-ETB repository top-level structure

2.1.1 Development directories

These directories contain the implementation source code, tests, build and deployment workspace and documentation for O-ETB.

2.1.1.1 src directory

This directory and subdirectories contain the Prolog source code modules comprising O-ETB. Annex X presents the complete list of source code modules in O-ETB v1.1.0.

2.1.1.2 doc directory

This directory contains README files as well as User Manual, Release Note, and design documents and diagrams.

2.1.1.3 BUILD directory

The BUILD directory contains the etb and etb-server executables built by the makefile in the src directory. It also contains other files related to build and deployment of O-ETB.

2.1.1.4 TEST directory

This directory contains O-ETB test cases, scripts, and results.

2.1.2 Database directories

These directories contain the data structures that represent the persistent but malleable state of an O-ETB database instance. These contain source information and derived artefacts for potentially multiple assurance and certification efforts that share a common (but non-fixed) knowledge base.

Figure 2 illustrates the structure of the database portion of the O-ETB repository.

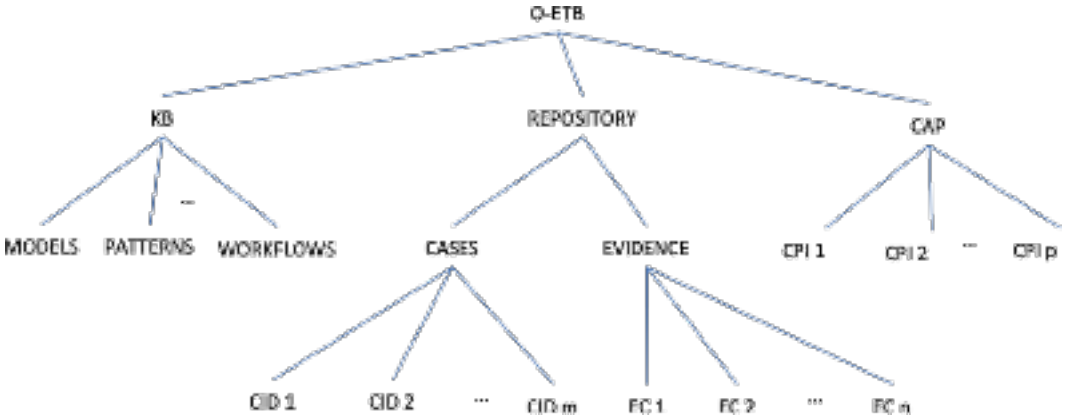

Figure 2 Database structure

2.1.3 Source Code

The static Prolog source code, including declaration of dynamic predicates, has been consolidated into a "src" sub-directory of the source repository. Module files have been added for new functionalities such as command line interpreter, command-invoked actions, command procedures and scripts, self-tests, and parameterization of the implementation.

A significant organisational enhancement of the source code in v1.1.0 has been the addition of a global parameters file ("param.pl") that consolidates O-ETB operational settings and constant definitions in one place complemented with corresponding references to the parameters at appropriate places in the source code.

2.1.4 Knowledge Base

A distinct place in the runtime file structure has been created for the knowledge base used by O-ETB ("KB" directory). The knowledge base includes the library of assurance case argument patterns ("PATTERNS" sub-directory), the library of system models ("MODELS" sub-directory), a library of workflow patterns ("WORKFLOWS" sub-directory), and in the future will be extended with other directories for additional expert knowledge applicable to the construction of assurance cases, evidence categories with corresponding evidential tools, and specifications for the requirements on the content and presentation for certification assurance packages.

2.1.5 Assurance case and evidence repositories

Under the "REPOSITORY" directory the assurance case repository ("CASES" sub-directory) and the evidence repository ("EVIDENCE" sub-directory) are located. Separate directories are created under CASES for each assurance case. The EVIDENCE directory is organized by directories corresponding to evidence categories and under those are sub-directories are used for individual evidence records.

2.1.6 Certification Assurance Package

The runtime file structure includes a default location ("CAP" directory) for the export of collections of artefacts tailored to specific certification targets.

2.2 'ETB' ASSURANCE TOOL

The 'etb' tool enables standalone AC pattern development, AC pattern instantiation and tool development and testing. It is also extended with the ability to start the etb server to run in the background. The etb tool provides the ability to work with a pattern as it is being created or modified, to test its instantiation algorithms and use of the knowledge base.

2.2.1 Command line options

When a compiled version of the O-ETB Assurance Tool is started from the command line as "etb", several command line options are recognized.

- `--token`, `-t` <epptoken> use the token to authenticate requests to the ETB
- `--port`, `-p` <portnumber> server should listen on specified port number
- `--selftest`, `-s` run self-tests when the tool is started before entering the interactive command interpreter
- `--verbose`, `-v` show detail for all messages when available
- `--jsonresp`, `-j` encode API responses in JSON

The `token` argument provides a shared secret that serves as protection against unauthorized interactions that could compromise the O-ETB data bases.

The `port` argument provides a port number to override the configured port number to be used when the server mode is invoked within the interactive command interpreter.

The `jsonresp` argument instructs the tool to respond to all API requests with JSON responses (when the server mode is invoked from within the Assurance Tool).

2.2.2 Assurance Tool interactive commands

The 'etb' Assurance Tool is a command driven application. After starting 'etb' it offers the prompt "etb>". There are basic general commands available in the normal mode (user) and an extended set of commands for use by a developer in development mode (advanced). The command interpreter is easily extensible. The command interpretation machinery and the generic commands are defined

in the command module. Commands for specific groups of functionalities can be defined in separate smaller files where the syntax, argument types, operational semantics and help text for such commands can be isolated. Commands that invoke ETB-specific functions are currently defined in a separate file command_etb.pl. It is anticipated that a set of commands for knowledge base or evidence management may be defined separately in the future. Entering the command "help" will list all the commands available in the current mode.

Table 1 shows the general purpose commands available in the ETB Assurance Tool.

Table 1 Assurance Tool general purpose commands

`advanced. | basic. | developer. | etb.`	Switch to advanced (basic, developer or etb) mode.
`halt.`	Exit the etb tool. (Will also terminate a server spawned from an interactive instance.)
`help.`	List the commands available in the current mode.
`help( <command name> ).`	Give a synopsis of the named command.
`proc( <proc name> [, step] ).`	Run the named stored procedure, optionally pausing after each command.
`proc( <proc name> [, verbose] ).`	Run the named stored procedure, optionally verbose.
`quit.`	Terminate etb command loop or script, but stay in Prolog top level.
`regtest.`	Run built-in regression tests.
`script( <file name> [, step] ).`	Run the named command file, optionally pausing after each command.
`script( <file name> [, verbose] ).`	Run the named command file, optionally verbose.
`selftest.`	Run built-in self tests.
`time( <etb command> ).`	Execute *<etb command>* and report time stats.
`time( <etb command>, <reps> ).`	Execute *<etb command>* the number of times indicated by *<reps>* and report total time stats.
`traceoff.`	Turn off tracing.
`traceon.`	Turn on tracing.
`traceone.`	Turn on tracing for only the next command.

`version.`	Display the current version number and version description.
`versions.`	Display past and current version numbers with descriptions.

Table 2 shows the data and control commands available in the ETB Assurance Tool.

Table 2 Assurance Tool data and control commands

`ac_export( <base name>, <format> ).`	Export the current assurance case to named sub-directory in the CAP directory.
`attach_case( <assurance case id> ).`	Attach the identified assurance case in the CASE repository.
`detach_case.`	Detach the currently attached assurance case.
`etb_reset.`	Perform reset of ETB databases including dynamically created persistent objects.
`etb_reset( <domain>, <name> ).`	Reset ETB database item *<name>* in database *<domain>*.
`etb_server.`	Initiate the ETB Assurance Server for background operation.
`etb_server( <arg> ).`	Initiate the ETB Assurance Server for background operation with the specified argument.
`etbt.`	Run currently pre-configured default ETB test.
`etbt( <test id> ).`	Run the identified ETB test.
`etbt( <test id>, <mode> ).`	Run the identified test in *<mode>*.
`etbtests.`	Run all ETB tests.
`import( <spec type>, <file>, <sid> ).`	Import the *<spec type>* specification from *<file>* and store in the KB as *<sid>*.
`instantiate_pattern( <pattern name>, <pattern arguments list>, <assurance case id> ).`	Instantiate a pattern from the KB into the CASES repository with the given case id.
`instantiate_pattern_list( <list of pattern_name-arguments_list pairs>, <assurance case id> ).`	Instantiate a list of patterns from the KB into the CASE repository with the given case id.
`load_model_v( <model id>, <policy var>, <platform var>, <config var> ).`	Load system model from the KB and instantiate the provided variables.
`update.`	Update CASES and EVIDENCE to incorporate asynchronously updated evidence.

It is expected that additional advanced user commands for development and diagnostics will be added in the future. The extensibility of the command interpreter facilitates this.

2.2.3 Timing Commands

The 'etb' Assurance Tool has two commands for timing execution of other 'etb' commands. The first, time(etb_command) times the execution of a single command. The second, time(etb_command, repetitions) executes the command for the number of times specified by the repetitions argument, and reports timing statistics. The second version turns off console output from the repeated execution of the command to eliminate the substantial timing impact of generating console output, leaving a result that is more representative of the processing time that would be consumed by a server to do the equivalent work load.

2.2.4 Command procedures and scripts

There are predefined command procedures ("procs") that run the examples and can be used for testing and demonstrations. At the "etb>" prompt a predefined procedure (e.g. named "myproc") can be run with the command `proc(myproc).` It can be run with verbose output with the command `proc(myproc,verbose).` It can be made to prompt and wait for user instruction to proceed (empty line input) with the command `proc(myproc,step).`

The file procs.pl that defines the predefined procedures. The procedures utilise the same commands available at the command prompt. The user may define additional procedures in the procs.pl file for subsequent execution as above.

A sequence of 'etb' commands can also be stored in a file, in which case it is referred to as a *script*. Scripts may be run with a `script` command, analogous to the `proc` command, with the script file name substituted for the stored procedure name. `verbose` and `step` are valid options also for the execution of scripts.

2.3 'ETB-SERVER' ASSURANCE SERVER

Note: This section describes features that are anticipated in a forthcoming O-ETB release but which are not yet committed. Thus, the information in this section should be regarded as preliminary or notional. It is provided early to elicit comments prior to implementation.

2.3.1 Server assurance case sessions

To be able to service requests to work on multiple assurance cases simultaneously, each user process communicating with the server to create, update or delete an assurance case must establish as session with the server. The associated session identifier will be associated with an assurance case in the repository. Further API requests specifying the session identifier will be processed in the context of the assurance case associated with the session identifier.

2.3.2 Server certification projects

The collection of artefacts needed for a certification are known to O-ETB as a named *certification project*, which is created and maintained under the CAP directory. It represents the current rendering, for a target certification effort, of relevant content from the CASES and EVIDENCE Repositories. It will typically grow and evolve over many sessions of assurance case and evidence development and evolution.

2.3.3 Assurance Server APIs

A programmatic interface, in the form of RESTful APIs, constitutes the Assurance Server Interface, as summarized in Table 3. Most of these APIs correspond to operations available as interactive (or scripted) commands in the Assurance Tool. Indeed, command procs or scripts may be invoked through the Server API.

Table 3 Assurance Server APIs

Server API Name	Arguments	Returns	Description
AC_Attach	AC Id	Status	Attach an assurance case contained in the CASES Repository
AC_Detach	none	Status	Flush and detach the currently attached assurance case
AC_Export	AC Id, Basename, Format	Status	Export the identified (or CURRENT) AC to the CAP with specified name and format
AC_Inst_Pattern	AC Pat Name, Pat Args, AC Id	Status	Instantiate a pattern from the KB into the CASES repository with the given AC Id

AC_Inst_Pattern_List	PatName-PatArgs List, AC Id	Status	Instantiate with corresponding args a list of patterns from the KB into the CASE repository with the given case id
AC_Update	AC Id	Status	Update CASES based on changes to EVIDENCE. Only specified AC Id, or ALL
CAP_Read	CAP Session, CAP Id	Id-Type list, Status	Read the contents of the CAP project associated with the session
CAP_Create	Auth Token, CAP Id, Cert Rqmts Reference	CAP Session, Status	Create a CAP project with referenced KB requirements and establish a CAP session
CAP_Delete	CAP Session,	Status	Delete the CAP project associated with the session
CAP_Session	Auth Token, CAP Id	CAP Token, Status	Establish a session attached to an existing CAP Project
CAP_Update (export)	CAP Token, Artefact kind, Artefact Id	CAP token, Status	Create a certification assurance package
COM_Proc	Proc Name	Execution transcript, Status	Run the named stored command procedure

COM_Script	File Name	Execution transcript, Status	Run the command script contained in the named file
COM_Version	N/A	Version number, description, status	Returns the version number of the responding server.
KB_ECat_Create	Evidence Category Id	Status	Create a new evidence category
KB_Agent_Create	Tool Id, Agent Executable File	Status	Associate a new evidence tool agent with an evidence category
KB_Model_Create	Model Id, Model Contents	Status	Create a new model entry in the KB
KB_Model_Read	Model Id	Model Structure, Status	Return contents of a model entry
KB_Model_Update	Model Id, Model Structure	Status	Update an existing model entry
KB_Model_Delete	Model Id	Status	Delete a model entry
KB_Pattern_Create	AC Pat Id, AC Pat Structure	Status	Create a new AC pattern
KB_Pattern_Read	AC Pat Id	AC Pat Structure, Status	Read the AC pattern from the KB
KB_Pattern_Update	AC Pat Id, AC Pat Structure	Status	Update an existing AC pattern

KB_Pattern_Delete	AC Pat Id	Status	Delete the identified AC pattern from the KB
KB_Property_Create	Property Id, Property Structure	Status	Create a new Property
KB_Property_Read	Property Id	Property Structure, Status	Read the Property from the KB
KB_Property_Update	Property Id, Property Structure	Status	Update an existing Property
KB_Property_Delete	Property Id	Status	Delete the identified Property from the KB
Server_Reset	Auth Token, Domain, Name	Status	Reset the named database item in the given database domain or complete reset for "ALL"
Server_			

2.4 ASSURANCE CASE PATTERNS

An *assurance case pattern* is a representation in an assurance case pattern language (e.g., [ ]) containing typed logical variables that can be instantiated producing a concrete term that is immediately translatable into an *assurance case fragment* in whatever representation is being used for assurance cases (such as the Goal Structuring Notation (GSN) [ ] or Structured Assurance Case Metamodel (SACM) [ ]).

The assurance case pattern language has a well-defined grammar that represents a structure of argumentation, evidence, and metadata that includes typed logical variables that can be instantiated resulting in another grammatically valid sentential form that may still include logical variables or a ground sentence representing an assurance case or part of an assurance case. A special type of logical variable in an AC pattern is of *Evidence* type (or a collection of *Evidence* types corresponding to the different classes of evidence produced by different evidential tools or processes carried out by persons).

Such variables become instantiated to terms that are referred to as *evidence specifiers*.

Evidence specifiers are treated like other ground terms in an assurance case fragment even though they may still contain variables the instantiations of which are deferred until the running of the evidential tools/processes that produce the ground instances of the evidence data. The running of such tools/processes is done by the O-ETB when it is asked to produce a concrete assurance case or to update an existing assurance case because of some change that may cause the values of the produced evidence data to change.

In summary, the O-ETB constructs assurance cases by executing workflows and instantiating assurance case patterns stored in its knowledge base (KB). The operation of the O-ETB can be tailored by supplying suitable workflows and assurance case patterns in the KB.

A textual language representation is easily constructed from a graphical representation of a GSN assurance case and vice versa. The present implementation of the language does not yet include some of the standard GSN structural abstraction extensions for AC patterns but these extensions are part of the currently proposed new requirements for O-ETB.

2.4.1 Basic syntax of pattern language

Top level of assurance case pattern

- <ac pattern> ::= ac_pattern(<pattern id>, <formal parameters>, <goal>)
- <formal parameters> is a list of parameter names
- <goal> ::= goal(<id>, <claim>, <context>, <body>)
- <claim> ::=
- <context> ::=
- <body> ::=

2.4.2 Parameterized elements in the pattern language

2.4.2.1 Pattern Parameters

2.4.2.2 Extensions

The GSN Community Standard introduces a graphical notation for these features but this requirement suggests a textual representation that is consistent with the graphical notation and considered adequate for O-ETB in the context

of MedSecurance. To that end, O-ETB will allow the following constructs in the target of GSN relations:

- alternative(<list of alternative elements>) – *optionality* – one or more (inclusive OR)
- alternative (<list of alternative elements>, <min>, <max>) – *optionality* – m of n, e.g. 1 of 3 (exclusive OR)
- optional(<element>) – *optionality* – 0 or 1 occurrence of the instantiated <element>
- multiple(<element>) – *multiplicity* – 1 or more occurrences of the instantiated <element>
- multiple (<element>, <cardinality>) – *multiplicity* – definite number of instantiated occurrences of <element>
- multiple (<element> , <low>, <high>) – *multiplicity* – indefinite number of instantiated occurrences of <element> ranging from <low> to <high>

These abstractions are applicable to all GSN relation types (i.e., SupportedBy and InContextOf).

There will also need to be some operations on "AC forms" (ACs that are at any stage of instantiation from uninstantiated pattern up to fully instantiated AC). Such operations include determining whether there are still uninstantiated terms, or finding uninstantiated terms, etc.

3. INSTALLATION, BUILD AND OPERATION

3.1 OVERVIEW

The executable O-ETB system is composed of two physical configuration items, the 'etb' Assurance Tool and the 'etb-server' Assurance Server. The Assurance Tool is used to develop and assurance case patterns and develop assurance cases. The ETB Assurance Tool and Server use those patterns to create instantiated assurance cases. These components share a common set of Prolog source modules and their separate executables are constructed using the `make` command with a simple ***makefile***, which accompanies the sources. A primary objective of the 'etb'/'etb-server' development is to create a lightweight and highly portable evidential tool bus framework that can be easily adapted to different situations and applications, that has a minimum of external dependencies, and that requires minimal resources to run. This objective has been achieved to a high degree in the current implementation.

3.2 PREREQUISITES

The 'etb' Assurance Tool and the 'etb-server' Assurance Server are implemented in the Prolog language and require the SWI-Prolog environment to run. This is the software's primary external dependency apart from the operating system and a small number of utilities. The version of SWI-Prolog used for the current version is version 8.2.4 available from www.swi-prolog.org.

SWI-Prolog is available for several operating environments, including Microsoft Windows (64 bit) and (32 bit), MacOS X 10.6 and later on Intel, and several Linux versions including Ubuntu. It is also available in Docker containers and as a source distribution that one can build locally.

Our O-ETB implementation uses *only* the libraries that come with the Prolog distribution or small open source utility packages that are included in the O-ETB repository. Furthermore, the functional architecture has been organized so as to place adaptation into the hands of application developers without requiring modification to the core implementation. Rather the implementation is adaptable through the contents of its O-ETB Knowledge Base.

The software is provided as a set of Prolog source files and/or as "executable" files that have the Prolog runtime environment already linked in. Prior to installing and building the software it is required that Prolog be installed. The exception to this requirement is if there is already a compiled version of 'etb' and 'etb-server' for the target platform, in which case all the dependencies are already linked into the executable.

3.2.1 Install prerequisite software

Install the following:

- SWI Prolog – Prolog language system and libraries, version 8.4.2 or later. SWI Prolog is available for several operating environments, including MacOS, Windows, and Linux. See (http://www.swi-prolog.org)
- gsn2x – rendering a textual representation of GSN as a graphic image – not yet used but expected to be used (https://github.com/jonasthewolf/gsn2x)
- graphviz / dot – graphical rendering of input provided in the textual dot notation (https://graphviz.org/download/)
- make – standard Unix environment build tool
- find – standard Unix environment tool

The environment must be configured so that the Prolog runtime can execute the other external software items listed above using "shell", "exec" or "forkexec".

3.2.2 Install the O-ETB source files and/or executables

The current version of the etb tool consists of a directory tree, presented in Figure 1 and Figure 2, including source files and example files. The complete configuration list is given in Section 8 Annex A. The distribution is provided as a zip file of this directory tree, in the project internal repository at MedSecurance/Internal/WPs/WP3/AssuranceWork/O-ETB-version-1.1.0, or as a GitHub repository.

3.3 BUILDING AND RUNNING THE 'ETB' ASSURANCE TOOL

The etb assurance tool is implemented in Prolog and requires the SWI Prolog environment to run. The etb tool can be provided as a set of Prolog source files and/or as an "executable" that has the Prolog runtime environment already bundled in. This executable is made by the makefile located with the source files that must be run in an environment that has SWI Prolog installed.

3.3.1 Compile the 'etb' policy tool

If a ready-made executable 'etb' has been provided it may be executed directly from a command shell prompt, resulting in the 'etb' prompt "etb>". If this is the case skip down to the next section. Otherwise, or if the provided executable is out-of-date, take the following steps:

In the source directory "src" execute the "make" command. It will use the provided makefile. This will (re-)create the 'etb' executable in the BUILD directory. Executing the newly built executable, as for a ready-made executable, will result in the prompt "etb>". The makefile also contains the targets clean, clean_cap and clean_repo. These can be used (as in make clean) to restore the CAP, the REPOSITORY or both to a fresh empty state. These make targets can be used whether the make or the manual method is used to compile and run the Tool.

Note that a compiled ‘etb’ executable may be invoked with the command line options detailed in Section 2.2.1. It is not necessary to use any of these options for the present purpose of demonstration.

Alternatively, the tool may be compiled and run manually within the Prolog interactive top-level command loop. The benefits of this approach are primarily for a developer or tester, that is, the ability to break out of the ‘etb’ command interpreter to the Prolog prompt to inspect data structures, to perform operations not provided as ‘etb’ commands, to use the interactive Prolog debugger, and to incrementally edit and recompile the code without affecting (mostly) the current data structures. The ‘etb’ command interpreter may be re-joined without complete reinitialization by entering the command ‘etb’ at the Prolog prompt. For convenience, the command “make”, which is a Prolog-level command, is also provided as an ‘etb’-level command so it is not necessary to exit the ‘etb’ command interpreter to incrementally recompile the running ‘etb’.

To compile manually, start SWI-Prolog from a command shell prompt using the name of the SWI-Prolog executable (usually ‘swipl’, ‘swi-pl’, or something similar, depending on how SWI-Prolog was installed).

After printing a short banner SWI-Prolog will display its prompt “?- “.

At the Prolog prompt enter “[etb].” (without the quotes but with the square brackets and the period). Note that this is different to entering “etb.” at the Prolog prompt, which executes, rather than compiles, ‘etb’. Entering the name “etb” within brackets tells Prolog to consult the file “etb.pl” which is the top-level file of the source. This results in compiling etb.pl and all of the module or included files that it references, and so on recursively.

Prolog will compile the code and print “true.”

Execute the code by entering at the Prolog prompt: “etb.”

Now you should see the ‘etb’ prompt “etb>”.

3.3.2 Test the installed ‘etb’ tool

The O-ETB Assurance Tool has some built-in self tests. These may be run to ensure that everything is working correctly after building ‘etb’. Follow the instructions to run the ‘etb’ tool. If you compiled manually in Prolog start it with the Prolog prompt command “etb(self_test).” This will run some built-in self-tests when it starts. Other alternatives to self_test are regression_test, no_initial, and verbose. The lists of self tests and regression tests are defined in the file param.pl. The no_initial option, which bypasses much of the initialization, and verbose are useful primarily to ETB developers. To not run any of these options simply start the tool with the Prolog prompt command “etb.”

The self-tests can also be run by starting 'etb' normally and entering at the 'etb' prompt the command "selftest."

Procedures made up of 'etb' commands may be predefined in the procs.pl file. Look at the ones there and try them by entering the etb command "proc(ProcName).", where ProcName is the name of one of the procedures defined in procs.pl.

3.3.3 Running the examples

There are several examples included with the sources of the O-ETB Policy Tool. These include examples described in documents and PowerPoint slide decks used to introduce the O-ETB concepts.

There are predefined procedures ("procs") that run the examples. At the etb> prompt a predefined procedure (e.g. named "myproc") can be run with the command `proc(<procname>).` It can be run with verbose output with the command `proc(<procname>,verbose).`

It is instructive to examine the file procs.pl that defines the predefined procedures. Each procedure definition consists of a list of the same etb commands available at the command prompt contained within the structure `proc(<procname>,[<command>, <command>, … ]).` The commands are given within the list without the terminating "." that is required when they are given interactively. However, each proc definition does end in ".".

See Section 5 for further details on the currently included examples. The user may define additional procedures in the procs.pl file for subsequent execution as above.

3.4 BUILDING AND RUNNING THE 'ETB-SERVER'

The O-ETB Assurance Server etb-server is implemented in Prolog and requires the SWI-Prolog environment to build and run. The server can be provided as a set of Prolog source files and/or as an "executable" that has the Prolog runtime environment already bundled in. This executable is created by the makefile located with the source files and the make command must be run in an environment that has SWI Prolog installed.

3.4.1 Install the O-ETB source files and/or etb-server executable

The current version of O-ETB consists of a directory tree including source files, prototype structure for the runtime data files and example files. The distribution is provided as a repository on GitHub.

3.4.2 Initiating the 'etb-server'

The O-ETB server may started from the 'etb' policy tool. Though it may be useful for development and debugging, in normal use the assurance server should be started with the compiled executable 'etb-server'. This is preferable since it allows the command line options to be specified.

If you do want to start the server from the policy tool follow the instructions above to get 'etb' running. After starting 'etb' it offers the prompt "etb>". There are a set of basic commands available in the normal mode (admin) and an extended set of commands for use by a developer in development mode (advanced). Entering the command "help" will list the available commands in the current mode. If you want to load any policy files, do it now with the 'etb' command "import(policy(PolicyFileName)).", where PolicyFileName is the name of a .pl file relative to the execution directory. You can also combine policies with the 'etb' "compose" command. When you have the desired policies loaded and composed, start the server from the 'etb' tool using the command "server(PortNumber).", where PortNumber is an unused TCP port. The server will be started and will be listening to that port for calls to its RESTful API. A server started in this way will expect the default admin token (the string "admin_token", without the quotes) in the policy administration API calls.

The 'etb-server' can be run in the background from a startup script. To the command 'etb-server' add any desired command line options and append a "&" to run the process in the background.

3.4.3 Test the installed 'etb-server'

There are a number of shell scripts of curl commands included with the source in the TEST subdirectory. These scripts can be run to send a sequence of requests to the server to test for known correct answers. Basic tests are in `servercurltest.sh`. (Note: These scripts run illustrative tests, they do not run exhaustive tests.)

A top-level script `run-nn-tests.sh` runs all of the numbered test scripts. Expected results are contained in corresponding numbered files.

3.5 EXTENDING ASSURANCE TOOL AND SERVER

The most common extension to the tool and server is to enable the integration of new external evidential tools into the O-ETB core workflow by creating an appropriate *tool agent*.

The 'etb' tool can be easily extended in several ways:

- Commands can be added by modifying the `command` module to add a `syntax`, `semantics` (optional), `help`, and `do` clause for the new

command. A `syntax` clause must be added for the command. This clause declares the command name and parameters, and what mode the command belongs to, admin or advanced. Admin commands are available in admin mode, but also accessible in the advanced mode but not vice versa.

- The self-test framework is implemented in the `test` module. Tests for specific new modules can be added in the `TEST` subdirectory. An example of a test definition file for the `etb` module is implemented in `TEST/etb_test.pl`.

- New predefined 'etb' command procedures can be added to the `procs` module. A `proc` clause is added for each new procedure to be defined. There are examples in the `procs.pl` file.

Global parameters are set in the `param` module. Settable parameters (those that can be changed from the 'etb' command line with the `set` command) are itemized in a list `settable_params`. For example, the parameter audit_logging is a settable parameter. Adding new settable parameters requires the new parameter name to be added to the list of settable parameters and to the `dynamic` directive above it in a fashion similar to the other entries.

Extending the 'etb-server' can be extended in several ways:

- Adding APIs

- Adding increased parallelism

Such server extensions tend to be more involved that just adding new commands or tests although adding APIs can be done my mimicking the structure of the code for existing APIs.

3.6 ASSURANCE TOOL & SERVER DEPLOYMENT FOR MEDSECURANCE

To be supplied with integration and deployment collaboration with Unparallel Innovation.

3.7 APPLICATION TO USE CASES AND BUILDING THE KNOWLEDGE BASE

Section 3.3.4 of the MedSecurance deliverable D3.1, Automated IoMT Security Analysis & Assurance Toolbox – Interim Version, describes the approach and strategy for developing the knowledge base (KB) of the Assurance Tool through its initial use on the project use cases (UCs). The goal is to build a KB that will enable the O-ETB to construct an assurance case (AC) and a certification assurance package (CAP) in a manner that is efficient, consistent, repeatable, preserves acquired knowledge, and memorializes the details of processes previously completed successfully.

The strategy comprises three stages:

Stage I – the initial stage consisting of manual interactive use, in which the models, properties, AC patterns, and certification requirements are developed and encoded in the KB. More about populating the KB is contained in Section 9 Annex B.

Stage II – the automation stage in which the models, properties, AC patterns and certification evidence requirements are used with the workflow automation and pattern instantiation provided to refresh the structure of the AC and re-generate certification artefacts when changes are made to system specifications, assurance methods or certification requirements. Using the AC patterns and models developed for the project UCs the AC will be re-instantiated and evidence re-generated/validated. This phase may be accompanied by some efforts to generalize the AC patterns library and other KB contents. The Assurance Server API and/or command scripts will be used to enable the greatest extent of automation possible.

Stage III – the post project stage in which the Assurance Tool, Assurance Server and the KB developed during the project, including efforts to generalize those results are applied, augmented and refined in future projects, and further enhancements are made to the Assurance Tool and Assurance Server implementation.

Refer to Section 3.3.4 of the MedSecurance D3.1 for a detailed description of the prerequisites, activities and results of each of the Stages introduced above.[1]

[1] D3.1 has confidentiality "Partners Only Distribution". Use case partners have access to D3.1. For other non-project general users the information from Section 3.3.4 will be replicated here in a future revision of this User Manual.

4. PRINCIPLES OF OPERATION

4.1 ARCHITECTURE

In this section we refer to the O-ETB functional architecture shown in Figure 3.

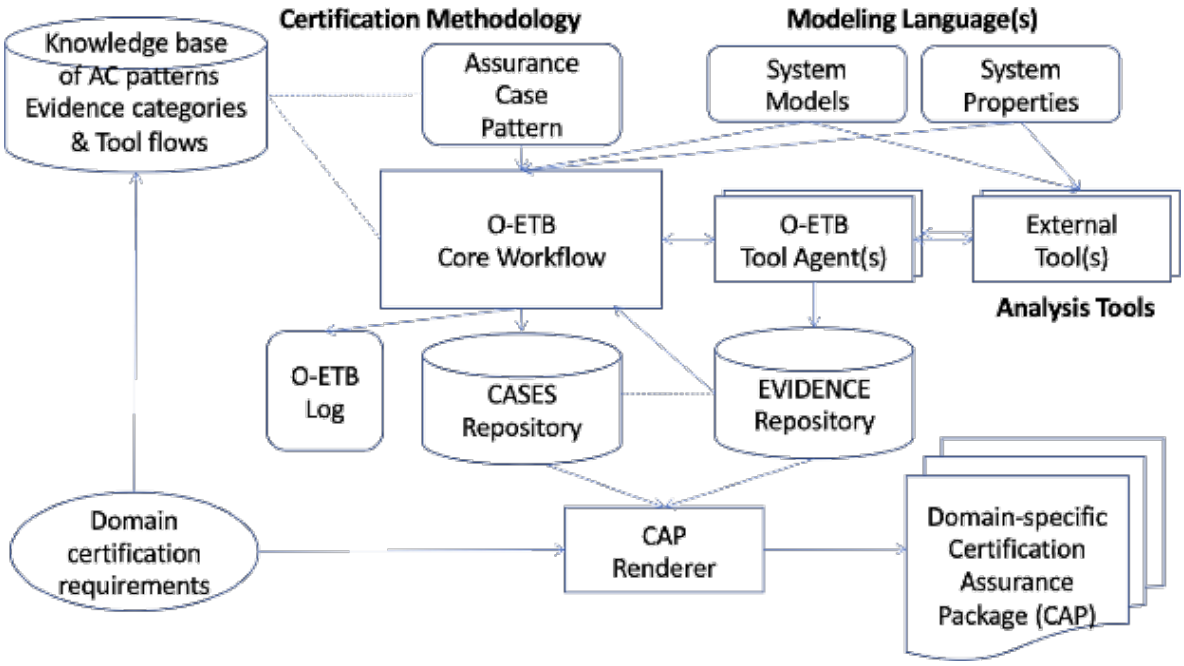

Figure 3 Functional architecture

4.2 O-ETB COMPONENTS

4.2.1 Functional components

4.2.1.1 Core Workflow Engine

Includes functions that generate instantiated assurance case patterns in the CASES Repository, creates evidence records needed to make the assurance case valid in the EVIDENCE Repository, invokes external evidential tools to validate evidence and to update evidence status in the EVIDENCE repository

4.2.1.2 Interactive Assurance Tool

4.2.1.3 Background Assurance Server

4.2.1.4 Tool Agent(s)

Invocation of executable wrappers for external evidential tools to interface them to O-ETB core workflow and the EVIDENCE Repository.

4.2.1.5 CAP Renderer

Functions to export assurance cases and evidence artefacts in a form to satisfy specific certification evidence presentation requirements.

4.2.1.6 KB Import

4.2.2 Data structures and stores components

4.2.2.1 Knowledge Base

Contains several categories of files containing assurance case argument patterns, workflows, system models and properties, evidence category-to-tool mappings, certification artefact requirements, etc.

4.2.2.2 Assurance CASES Repository

Persistent storage of dynamic assurance case elements.

4.2.2.3 EVIDENCE Repository

Persistent storage of evidence records with status and evidential tool results.

4.2.3 Packaging of core functions

The O-ETB Assurance Tool and Assurance Server represent two different modes of access to common underlying functionalities of the components identified above, that provide for the construction of assurance cases, including their supporting evidence, and the rendering of these assurance cases with their supporting evidence in a form appropriate for independent evaluation.

The Assurance Tool provides a command-based interface that can be used interactively and/or in a batch mode to run command scripts. The Assurance Server provides programmatic access to the core functions through an API. Both may be run on a shared centralised file structure, or, for development, testing or experimentation, run on independent instances of the file structure, as determined by configuration options and invocation arguments. As part of that file structure, assurance cases and evidence are constructed incrementally and stored persistently in the CASES and EVIDENCE Repositories respectively.

Note: The Assurance Server is a work-in-progress and not yet operational for the first prototype.

4.2.3.1 Certification Assurance Package

4.2.3.2 O-ETB Log

4.3 COMPONENT INTERACTIONS

Interaction among O-ETB components is illustrated in the sequence diagram in Figure 4.

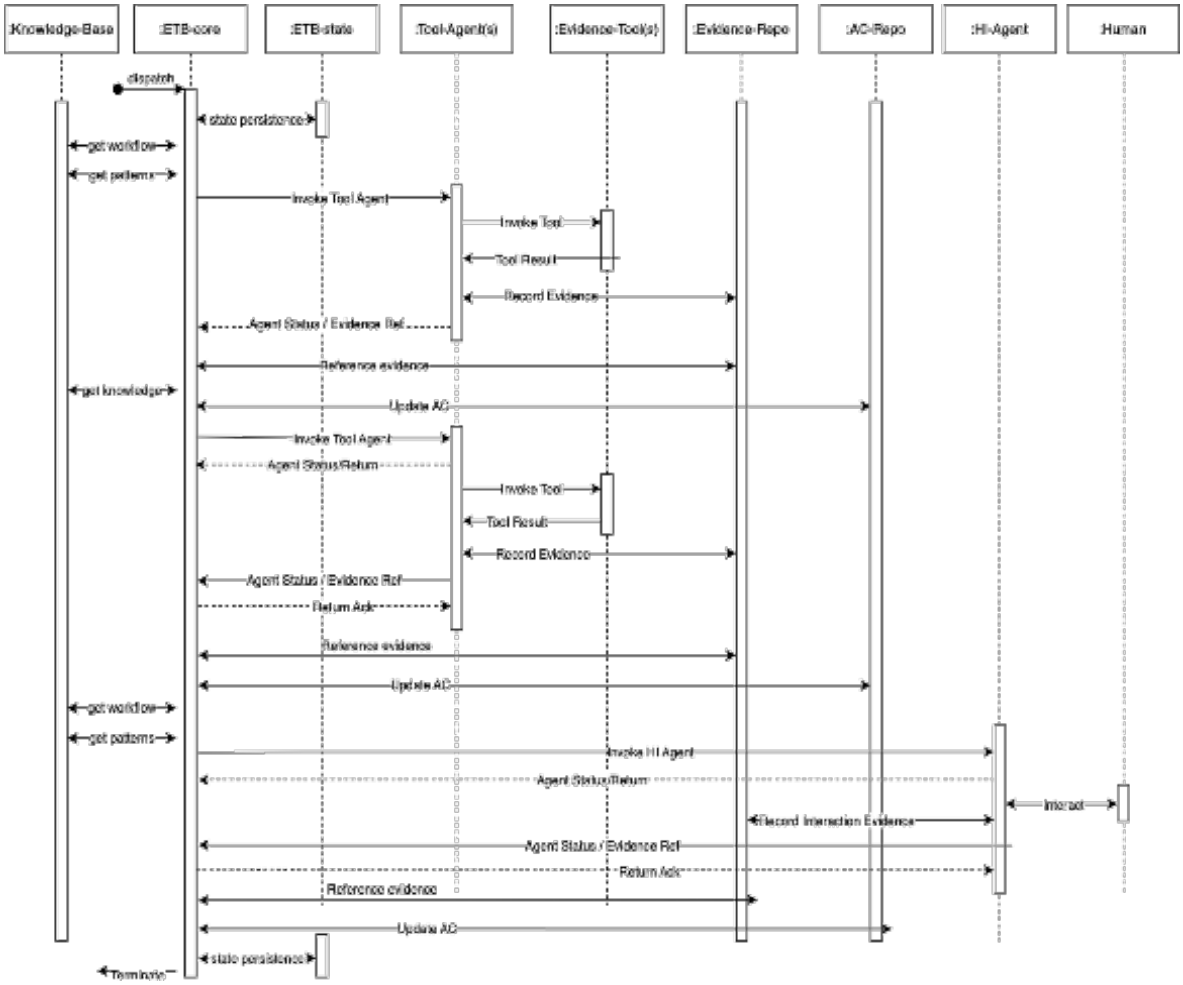

Figure 4 Component interaction sequence

Figure 5 illustrates the integration of external tools by tool agents.

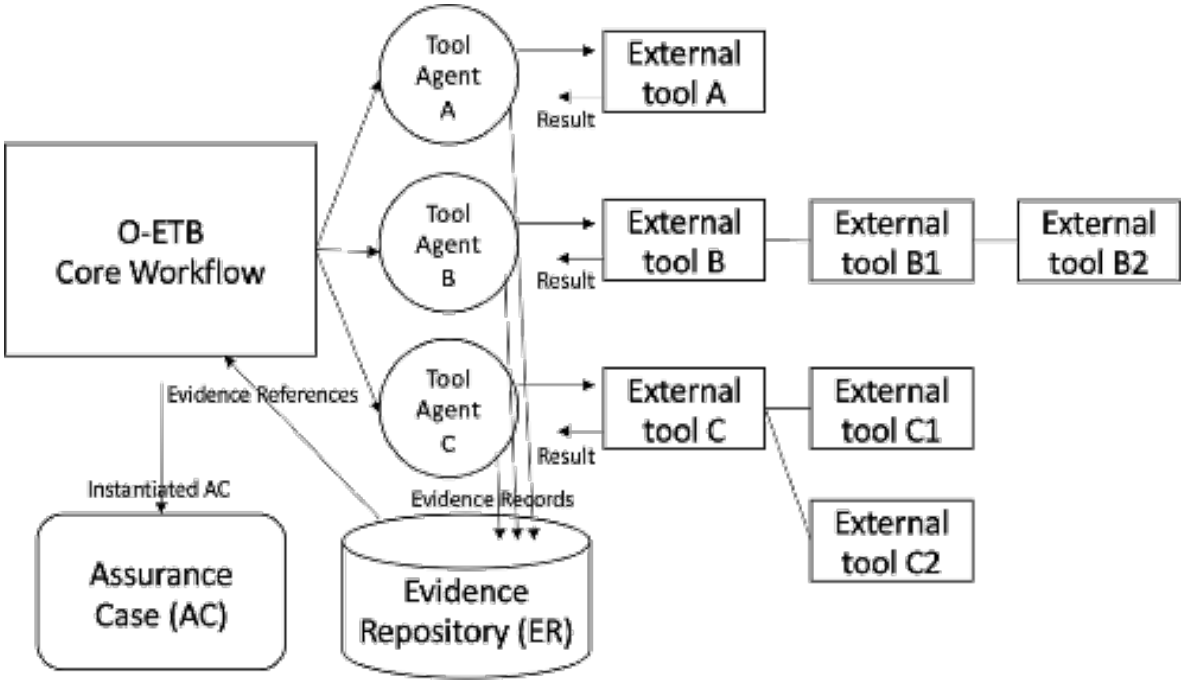

Figure 5 Tool agents integrate external tools

4.4 OPERATING MODES

A summary of the major modes and actions is shown in Figure 6.

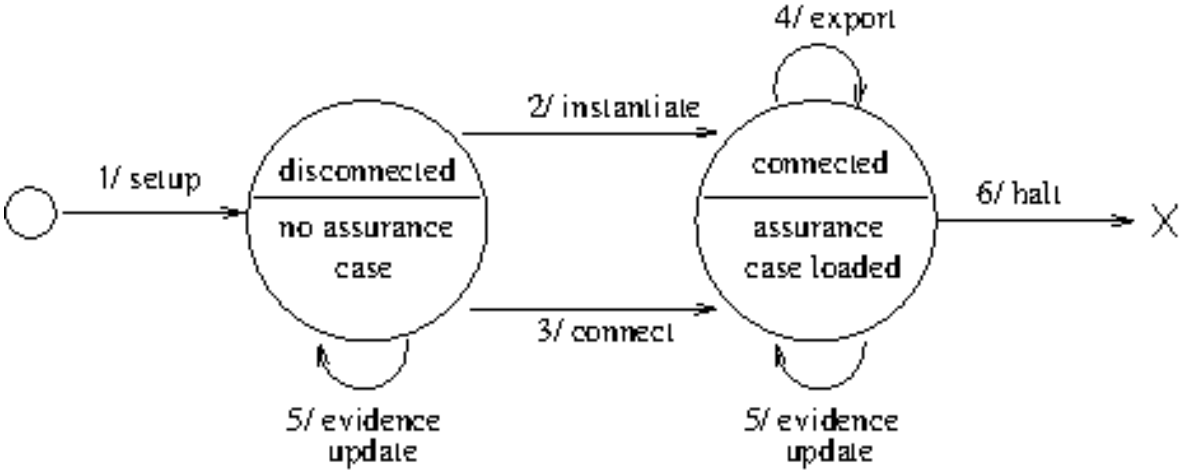

Figure 6 Major modes and actions

4.5 ARGUMENT PATTERN INSTANTIATION

O-ETB workflow is driven primarily by assurance case argument pattern instantiation. Construction of the argumentation and the generation of the associated supporting evidence is accomplished by the instantiation of a chosen top-level pattern using information from a specified system model.

Figure 7 provides the definition of the pattern instantiation rewrite rules.

$$[\ \underline{\text{pattern}}(\ Id,\ Args,\ Goal\ )\ ]\!]_{\mathcal{M},\mu} \Rightarrow [\ Goal\ ]\!]_{\mathcal{M},\mu/Args}$$

$$[\ \underline{\text{goal}}(Id,\ Claim,\ [\ Context,\ \ldots\ ],\ [Support,\ \ldots]\ ]\!]_{\mathcal{M},\mu} \Rightarrow$$
$$\underline{\text{goal}}(Id\#\mu,\ \ [\ Claim\ ]\!]_{\mathcal{M},\mu},\ [\ \ [\ Context\ ]\!]_{\mathcal{M},\mu},\ \ldots\ ],\ [\ \ [\ Support\ ]\!]_{\mathcal{M},\mu},\ \ldots])$$

$$[\ \underline{\text{context}}(Claim)\ ]\!]_{\mathcal{M},\mu} \Rightarrow \underline{\text{context}}(\ [\![\ Claim\ ]\!]_{\mathcal{M},\mu}\ )$$

$$[\ \underline{\text{assumption}}(Claim)\ ]\!]_{\mathcal{M},\mu} \Rightarrow \underline{\text{assumption}}(\ [\![\ Claim\ ]\!]_{\mathcal{M},\mu}\ )$$

$$[\ \underline{\text{justification}}(Claim)\ ]\!]_{\mathcal{M},\mu} \Rightarrow \underline{\text{justification}}(\ [\![\ Claim\ ]\!]_{\mathcal{M},\mu}\ )$$

$$[\ \underline{\text{strategy}}(Claim,\ [Context,\ \ldots],\ [Goal,\ \ldots])\ ]\!]_{\mathcal{M},\mu} \Rightarrow$$
$$\underline{\text{strategy}}(\ \ [\ Claim\ ]\!]_{\mathcal{M},\mu},\ [\ \ [\ Context\ ]\!]_{\mathcal{M},\mu},\ \ldots],\ [\ \ [\ Goal\ ]\!]_{\mathcal{M},\mu},\ \ldots\ ])$$

$$[\ \underline{\text{strategy}}(Claim,\ Iterator,\ [Context,\ \ldots],\ [Goal,\ \ldots])\ ]\!]_{\mathcal{M},\mu} \Rightarrow$$
$$\underline{\text{strategy}}(\ \ [\ Claim\ ]\!]_{\mathcal{M},\mu},\ [\ \ [\ Context\ ]\!]_{\mathcal{M},\mu},\ \ldots],\ [\ \bigcup_{\mu':SArgs\to\mathcal{M}}\ [\![\ Goal\ ]\!]_{\mathcal{M},\mu\sqcup\mu'},\ \ldots\ ])$$

$$[\ \underline{\text{evidence}}(Cat,\ Claim,\ [\ Context,\ \ldots]\ )\ ]\!]_{\mathcal{M},\mu} \Rightarrow$$
if call Agent@$Category(\mathcal{M},\ \mu)$ **==** (**valid**, $xref$) **then**
$\underline{\text{evidence}}(\ \ [\ Claim\ ]\!]_{\mathcal{M},\mu},\ [\ [\![\ Context\ ]\!],\ \ldots],\ xref)$
else $\perp$

$$[\ \underline{\text{goal-ref}}(Id)\ ]\!]_{\mathcal{M},\mu} \Rightarrow \underline{\text{goal-ref}}(Id\#\mu)$$

$$[\ \underline{\text{pattern-ref}}(Id,\ ActArgs)\ ]\!]_{\mathcal{M},\mu} \Rightarrow \underline{\text{goal-ref}}(\ GId\#\ \ [\ ActArgs\ ]\!]_{\mathcal{M},\mu}\ ),\ \textbf{eval}\ \ [\ Pattern\ ]\!]_{\mathcal{M},\mu}$$
(where $Pattern$ is the pattern identified by Id and GId is its top goal identifier)

Figure 7 Pattern instantiation rewrite rules

The following notational conventions are used:

- $A \Rightarrow C$ is a rewrite rule where A represents the sentential form to be rewritten and C the resultant sentential form.
- $[\![\ \mathcal{Z}\ ]\!]_{\mathcal{M},\mu}$ denotes the instantiation of an assurance case element $\mathcal{Z}$ with respect to the model $\mathcal{M}$ and using the mapping of formal parameters μ.
- Underlined symbols (e.g. <u>context</u>) are terminal symbols of the pattern language.
- Symbols with an initial capital letter (e.g. *Claim*) are logical variables that stand for a term, the type of which is intended to be inferred from the intuitive connotation of the ordinary meaning of the capitalized symbol.

- Bold symbols (e.g. eval or if then else represent operational semantics.
- An *Iterator* is one of:
    - iterate(*Name*, *Category*, list(*Values*))
    - iterate(*Name*, *Category*, expand(*ActArgName*))
    - iterate(*Name*, *Category*, subjects(*PolicyArgName*))
    - iterate(*Name*, *Category*, ss_flows(*PolicyArgName*))
    - iterate(*Name*, *Category*, nodes(*PolicyArgName*))
    - iterate(*Name*, *Category*, modes(*Subject*))
    - iterate(*Name*, *Category*, transitions(*Subject*))

When the instantiation procedure commences the top-level pattern given to the instantiation procedure becomes the current sentential form to be instantiated. Instantiation proceeds by left-to-right recursive descent on the structure of the form, triggering specific evidence generating tool agents when evidence (leaves) are encountered during pattern instantiation.

Two cases require a specific treatment beyond the (standard) recursive development:

- The instantiation of an invoked goal (goal-ref) requires the additional instantiation of that goal in the calling context, with the current evaluation of parameters. That is, the instantiation is not limited to the pattern itself but triggers additional instantiation of all other encountered sub-patterns.
- The instantiation of an evidence leaf node requires interaction with the agent associated with the evi- dence category. When the evidence is valid, it is included in the final result. Otherwise, the assurance case will remain incomplete and the missing evidence is reported to the user.

This presentation of the instantiation procedure as a set of rewrite rules has been kept abstract for the sake of exposition. A number of straightforward extensions are however possible, such as the following, that will be considered as the implementation of the O-ETB core workflow evolves:

- memoization of intermediate instantiation results, in particular, to avoid the costly re-generation of the same piece of evidence, referred many times in the assurance cases,

- versioning of segments of the assurance case to work hand-in-hand with memoization to reduce assur- ance case maintenance recompute overhead,

- concurrent execution, that is, allow instantiation to proceed concurrently for independent sub-goals and/or evidence. Such an option might ensure a faster construction process in particular when non- automatic (human-in-the-loop) and/or compute-intensive verification agents are used to produce and check specific evidences,

- additional fine-grained and expressive workflow notation and control mechanisms i.e., choose alterna- tives based on metrics, depart from depth-first instantiation, exploiting parallelism, restricting or allo- cating execution for some overall time budget, threshold amount of missing evidence, etc.

4.6 CASES REPOSITORY

4.6.1 Assurance case directories

The CASES repository is organised by assurance case identifier.

4.6.2 Persistent assurance case storage

The Assurance Tool and the Assurance Server are designed to build and maintain an assurance case incrementally because there are many distinct assurance activities that contribute to the construction of the assurance case and these activities may be performed over an extended period of time. Intermediate results and even incomplete results must be capable of being captured in one run and to be picked up and used to continue the effort by another run.

O-ETB uses a feature of the Prolog implementation to create, manipulate and store a database of such results and evolving structures. The SWI Prolog library ***persistency*** provides persistent storage for one or more dynamic predicates. O-ETB employs this feature for several persistent stores, including the CASES Repository and EVIDENCE Repository. O-ETB modules that are responsible for these databases declare the terms that can be placed in the database using the directive ***persistent***. By convention and implementation the declaring module is the sole agent having access to the persistent database and provides an internal API to access the database from other modules of the O-ETB system. As there may be multiple instances of the module executing in distinct instances of the Tool or Server, some operations require locking of critical sections in the O-ETB implementation.

4.6.3 Working assurance case cache

Each instance of the Assurance Tool or the Assurance Server has a working store for the currently attached assurance case. This cache represents the in-

memory copy of the declared persistent predicates and may be synchronized with the persistent CASES Repository, the current assurance case can be detached, and the cache cleared and reloaded with a different assurance case from the Repository.

4.7 ASSURANCE EVIDENCE

Evidence is data that provides information about some development artefact (specification, program code, tests, etc) that supports a goal or claim concerning key properties of the artefact that render it fit for purpose. Properties may include security, safety, privacy, functional correctness, performance, etc. Evidence results from performing some assurance activity and capturing the results to be referenced by argumentation for the related goal or claim. Typical assurance activities include:

- Formal verification that system models (which may include formal specifications and/or source code) satisfy formally specified properties or exhibit a required behaviour when executed (actually or symbolically).

- Verifying, by inspection and testing by independent experts, that an artefact or an entire system conforms to applicable and mandated standards.

- Verifying that the specific operating configuration of a system and its deployment and operating environment are consistent with the constraints and assumptions that have been imposed by the development and implementation.

- Analysis of the system architecture, components, and environment to discover potential faults and failures and to determine that appropriate steps have been taken to prevent or to mitigate the effects of such faults and failures. Such analyses may emphasize safety or security or some other characteristic that is essential for the system.

4.7.1 Evidence Validation

The assurance processes utilised by MedSecurance involve diverse assurance activities that create concrete evidence artefacts. The collection of evidence artefacts, along with the assurance case that organizes, relates, and presents the evidence, serves as a basis for belief that claims made by the assurance case are valid.

4.7.2 Evidence artefacts

An evidence artefact is generated by one of multiple potential sources, such as:

- Inspection by a person to confirm a claim such as conformance by a development artefact to an industry standard or a project guideline.

Such an inspection action may be entirely manual or may be assisted by one or more tools employed by the inspector that have not been integrated with the automated assurance process.

- A demonstrated and confirmable history of reliable use of a tool for a purpose similar to that for which the tool is being employed to create or support evidence directly used for the present assurance effort.

- Review by qualified experts to confirm that given facts and observations about the entity being assured support a conclusion that is in agreement with a target claim.

- Execution of a test case or collection of test cases in a particular test scenario and configuration to confirm that an expected result is produced.

- Execution of an analysis tool on one or more development artefacts using a specific set of situation-dependent input arguments to confirm that the tool produces an expected result or result that supports a claim.

4.7.3 Evidence categories

As discussed previously, for each argument pattern claim or sub-claim requiring substantiation one or more categories of evidence are identified that are capable of supplying the needed substantiation. Each evidence category is associated with a method or tool that is used to produce a concrete evidence artifact of the associated category given the inputs defining the current instance.

The results of performing the method or executing the tool are subject to an interpretation to determine that the result does not represent an exception and that its value is valid and sufficient to support the claim to which it is to be applied.

4.7.4 Evidential tool agents

O-ETB constructs assurance cases that are dependent upon external assurance activities and tools that produce the evidence referenced by the assurance case argumentation. ACs are modular in that they may include assurance goals that are not yet shown to be achieved by the existence of valid evidence. A complete and valid subtree of the AC, for which all needed evidence is complete, can serve as evidence for another goal. Only such reasoning, and tautological reasoning, is performed by the O-ETB itself. For all other evidence, it depends on external processes and tools. Each tool provides a certain class of evidence. When an item of evidence is required to make an element of the AC valid the specified class of the evidence is used to choose a tool agent corresponding to that class.

A tool agent manages the interaction of the O-ETB internal workflow with an external evidence-producing tool. Each tool agent communicates to the core workflow engine through a commonly defined interface and with the evidence repository through its own commonly defined interface. The tool agent embodies the interaction protocol with the specific tool or tool cluster that produces the needed class of evidence. The tool agent marshals the needed arguments, invokes the tool, interprets the tool's response and reports to the core workflow after storing the evidence in the evidence repository. The tool agent is responsible determining success of the evidence validation. That is, the agent must determine that the tool output is consistent with the result needed to satisfy the current goal.

The tool agent is required to perform the following actions:

- marshal the inputs and arguments to the tool in a form the conforms to the requirements of the tool.
- invoke the tool with the collected inputs and arguments.
- monitor the execution and/or receive the results of the tool invocation.
- recognize exceptions raised during the execution of the tool and take appropriate action.
- parse the results and interpret them to the extent necessary to determine that they conform to the expected format, that they do not represent an error condition, and that they appear to represent a valid outcome. The validation status is recorded appropriately as 'valid' or 'invalid'.
- store the outcome in an evidence record of the EVIDENCE Repository in a canonical form that has been defined for the corresponding evidence category, and the status is recorded.
- depending on whether the tool agent was launched for synchronous, asynchronous or silent operation return to the O-ETB core workflow, signal completion to the O-ETB server through its API, or silently exit.
- when the validation status of a tool invocation or manual assurance activity cannot definitively be reported as 'valid' or 'invalid' by the tool agent it is reported as 'ongoing'.

For evidence checking that reports a validation status of 'ongoing' the validation activity continues asynchronously with respect to O-ETB (for example as a manual process carried out by people). In such cases, once a definitive verdict of 'valid' or 'invalid' is reached the validation status file is updated either by the asynchronously running tool agent or by the Server's workflow program upon notification through the API.

In some cases, a group of external tools work in concert under the coordination of, and invoked by, a control process. In such cases the tool agent is designed to interface only with the control process and to leave the sequencing and communication with and among the other tools to the control process.

Evidence categories that correspond to assurance activities performed manually by one or more people, whether or not the activity is supported by tools outside of the O-ETB workflow, are included in the O-ETB workflow through special UI agents that deliver the relevant facts to a designated responsible individual who will in turn respond to the UI agent or programmatically invoke the Server API to report results of the assigned activity.

4.7.5 Evidence artefact validation status

The previous paragraph alludes to the validation status of an evidence artefact. The validation status is contained within a status file that is created within the EVIDENCE Repository when the artefact is initially created. Upon creation the status is recorded as 'pending'.

4.7.6 Re-synchronisation

EVIDENCE Repository updates by asynchronous tool agents do not immediately result in update to assurance cases that are supported by the evidence artefacts. Rather, update of assurance cases occurs when manually initiated by the user of the Assurance Tool or by the Assurance Server either periodically or when prompted through an API request.

4.8 EVIDENCE REPOSITORY

4.8.1 Evidence category directories

Evidence records of a particular evidence category are stored in a subdirectory named for the category.

4.8.2 Persistent evidence storage

A Prolog feature for persistent data bases is used for the evidence storage. The Prolog program manipulates a copy of the data in memory which is automatically synced to a file in the file system for persistence.

4.8.3 Working evidence cache

The running Assurance Tool or Assurance Server maintains a local working cache of evidence records that are backed by the persistent EVIDENCE Repository.

4.9 CERTIFICATION ASSURANCE PACKAGE (CAP)

The certification assurance package is a collection of artefacts, rendered from data in the CASES and EVIDENCE Repositories, into a form that is suitable

for presentation as part of an application for certification under a particular certification regime.

4.9.1 Tailoring the CAP to certification requirements

Figure 8 illustrates how domain-specific certification regime requirements can be used to control the evidence generation processes, the operation of the ETB, the structure of the assurance case and the final rendered content of the CAP for a specific certification regime or project.

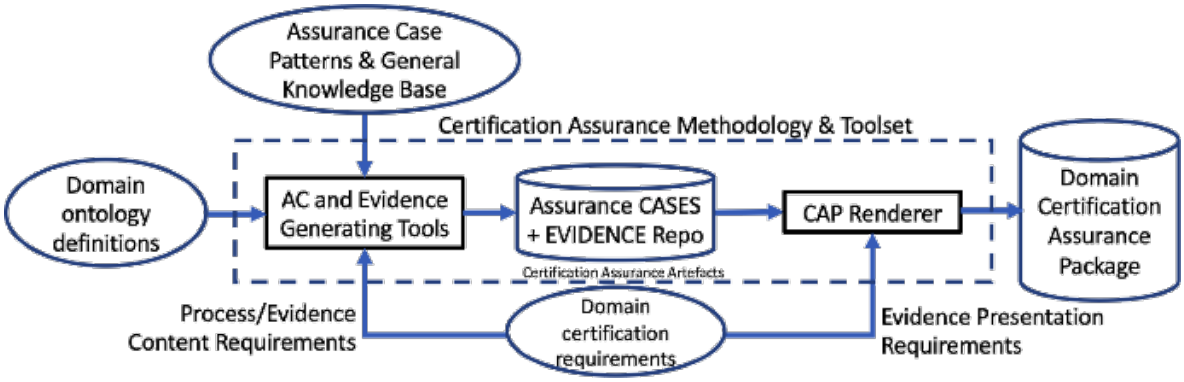

Figure 8 Tailoring the CAP to certification requirements

4.9.2 Organising multiple certification projects

Currently the staging area for certification assurance artefacts is a single directory named "CAP". Since the export operations of the O-ETB enable the file and directory names for the exported items to be specified, a user-chosen naming convention can assist in managing the contents of the CAP directory. Notwithstanding this possibility it would be useful to provide organisation within the CAP directory as a standard feature of O-ETB and this is anticipated as a future extension. This extension would include commands and APIs that provide the ability to register a certification project. The certification registry could be maintained as persistent database at the top-level of the CAP directory. Then, using the certification project identifier as a key, the maintenance of the process and presentation requirements of the certification could be stored in the Knowledge Base and the suitably presented artefacts could be organized in the CAP by subdirectories named for the project identifiers.

4.9.3 Text file artefacts

Assurance cases can be exported in textual form in a .txt file of specified name.

4.9.4 HTML directory artefacts

Assurance cases can be exported in graphical form as a collection of HTML files in a specified directory name. An ordinary web browser may be used to navigate the AC.

5. ASSURANCE TOOL AND SERVER EXAMPLES

Several examples of the use of the Assurance Tool have been encoded as procs in the file procs.pl. To gain familiarity with interactive use of the tool the commands defining these procs may be entered interactively one-at-a-time. When this is done each command must terminate with a "`.`".

5.1 EXAMPLE 1 – O-ETB TOOL INSTANTIATING A SIMPLE PATTERN

This example is encoded as the ex1 procedure in procs_etb.pl as shown in Table 4.

Table 4 Example 1 – instantiating a simple pattern

```
proc(ex1, [
        instantiate_pattern(person, ['Marius', 'Programming'], ac1),
        ac_export(ac1,txt),
        detach_case
]).
```

5.1.1 Instantiating an argument pattern

In Example 1 the command instantiate_pattern causes the pattern person to be instantiated with the arguments given in the list and will be stored in the CASES Repository as ac1.

5.1.2 Exporting assurance case as text

In Example 1 the command ac_export causes the assurance case in the working cache to be exported to the CAP directory as text in the file ac1.txt.

5.2 EXAMPLE 2 – ASSURANCE TOOL EXPORTING AN ASSURANCE CASE

This example is encoded as the ex2 procedure in procs_etb.pl as shown in Table 5.

Table 5 Example 2 – exporting an assurance case

```
proc(ex2, [
        update,
        attach_case(ac1),
        ac_export(ac1,html),
        detach_case
]).
```

5.2.1 Attaching an existing assurance case

In Example 2, after causing any pending updates to the Repository, the assurance case ac1 is attached and read into the working cache.

5.2.2 Exporting assurance case as HTML

In Example 2 the command ac_export causes the assurance case in the working cache to be exported to the CAP directory as HTML in the directory ac1.

5.2.3 Browsing an exported assurance case

An ordinary browser can be used to open the .index file in the directory created by the ac_export command above.

5.3 EXAMPLE 3 – ASSURANCE TOOL LOADING A MODEL AND INSTANTIATING A PATTERN LIST

This example is encoded as the ex3 procedure in procs_etb.pl as shown in Table 6.

Table 6 Example 2 – instantiating a pattern list from a model and exporting as HTML

```
proc(ex3, [
        set_v(ModelId, '1.0'),
        set_v(CaseId, ac2),
        load_model_v(ModelId, Policy, Platform, _Configuration),
        set_v(AC, [
                'foundational_plane'-[Platform],
                'operational_plane'-[Policy],
                'person'-['Alice', 'AC Patterns Definition'],
                'person'-['Bob', 'ETB Development]
        ]),
        instantiate_pattern_list(AC, CaseId),
        ac_export(CaseId, html),
        detach_case
]).
```

5.4 EXAMPLE 4 – MEDSECURANCE RPMS EXAMPLE

This example is a commentary on approaches to the use of O-ETB within the Remote Patient Monitoring System example given in MedSecurance D3.1.

There are several possible usage scenarios, depending on the IoMT system developer's workflow, availability and choice of tools, the extent of the implementation of O-ETB features at the time, the extent of experience gained in using MedSecurance methodology, the extend of experience gained with MedSecurance analysis tools, and the encoding of that experience as assurance case patterns and tool application patterns.

- Scenario 1 – manual encoding of O-ETB model data from models developed with modelling tools
- Scenario 2 – (semi-)automatic encoding of O-ETB model data from models

- Scenario 3 – no pre-existing and complete assurance case pattern for a particular target certification
- Scenario 4 -

Other factors that will affect the approach:

1. Existence of one or more general architectural models applicable to the particular RPMS example
2. Existence of automated extraction/translation of O-ETB model and property information from MedSecurance modelling tool outputs.
3. Existence of completed assurance case patterns embodying the MedSecurance methodology and targeting a particular target certification.
4. Existence of tool agents to automate all tool runs.
5. Availability of sufficiently mature versions of all modelling and analysis tools.

6. REFERENCES

[1] Medical Device Coordination Group (MDCG), https://ec.europa.eu/transparency/expert-groups-register/screen/expert-groups/consult?lang=en&groupID=3565

[2] Interfaces and Workflow Definition for AM-ETB, CITADEL Project D5.1, May 2017.

[3] AM-ETB Tool Bus for Tool Integration and Assurance, CITADEL Project D5.2, May 2018.

[4] CITADEL project, http://www.citadel-project.org, 2019.

[5] ISO/TC 215 CD 81001-2-1 Assurance cases, draft for comment, 18 December 2023.

[6] N. Shankar, et al, DesCert: Design for Certification, SRI International, arXiv:2203.15178, March 2022.

[7] Goal Structuring Notation Community Standard Version 3, The Assurance Case Working Group (ACWG), SCSC-141C, May 2021.

[8] Y. Matsuno, Design and Implementation of GSN Patterns: A Step toward Assurance Case Language, In-formation Processing Society of Japan, IPSJ Transactions on Programming 7(2):1-10, 2014.

[9] S. Gregory and M. Paschali, A Prolog-based language for workflow programming, University of Bristol, technical report CSTR-07-001, February 2007; and Proceedings of the 9th International Conference on Coordination Models and Languages, June 2007.

[10] Object Management Group (OMG), Structured Assurance Case Metamodel (SACM), Version 2.0, 2018.

[11] Medical Device Coordination Group, "MDCG 2019-11 Guidance on Qualification and Classification of Software in Regulation (EU) 2017/745 – MDR and Regulation (EU) 2017/746 – IVDR" Oct. 2019.

[12] Medical Device Coordination Group, "MDCG 2019-16 Rev.1 Guidance on Cybersecurity for medical devices," 2020.

[13] MedSec-Sec Control Taxonomy v2, https://github.com/MedSecurance/Internal/tree/main/WPs/WP3/OntologyWork/MedSec-Sec_Control_Taxonomy_Version 0.2.xlsx .

[14] J. Rushby, An Evidential Tool Bus. In Formal Methods and Software Engineering, 7th International Conference on Formal Engineering Methods (ICFEM), 2005.

[15] N. Selviandro, Assurance case pattern using SACM notation. In 2021 9th International Conference on Information and Communication Technology (ICoICT), pp. 494-499. IEEE, 2021.

[16] R. Wei, T. P. Kelly, X. Dai, S. Zhao, and R. Hawkins, Model based system assurance using the structured assurance case metamodel. Journal of Systems and Software, 154:211-233, 2019.

[17] T. A. Beyene and Harald Ruess, Evidential and Continuous Integration of Software Verification Tools. Proceedings 22nd International Symposium, FM, FLoC, 2018.

7. GLOSSARY OF TERMS AND ABBREVIATIONS

Table 7 Terms and Abbreviations

AC	Assurance Case
A-G	Assume-Guarantee
Assurance case argument pattern	A parameterized expression representing logical relationships among goals (or claims) and evidence, expressed in a form of GSN with extensions for structural abstraction
CR	CASES Repository
CAP	Certification Assurance Package
CWE	Core Workflow Engine
ER	EVIDENCE Repository
ETB	Evidential Tool Bus
GSN	Goal Structuring Notation
KB	Knowledge Base (O-ETB component)
LTL	Linear Temporal Logic
O-ETB	Open ETB
OMG	Object Management Group
SACM	Structured Assurance Case Metamodel – a standard from OMG for expressing assurance cases and evidence
TA	Tool Agent components

8. ANNEX A: O-ETB CONFIGURATION IDENTIFICATION

8.1 FILE IDENTIFICATION

The directory tree of the O-ETB v1.1.0 source repository and distribution is given in Table 8. A description of the contents and purposes of these directories and files is covered in Section 2.1 and a description of the contents of the files containing source code modules in Section 8.2.

Table 8 O-ETB v1.1.0 directory tree

./REPOSITORY/EVIDENCE	./src/stringutil.pl
./REPOSITORY/EVIDENCE/certificate	./src/models_api/configuration.pl
./REPOSITORY/EVIDENCE/certificate/README.md	./src/models_api/policy.pl
./REPOSITORY/EVIDENCE/axiom	./src/models_api/model.pl
./REPOSITORY/EVIDENCE/axiom/README.md	./src/models_api/platform.pl
./REPOSITORY/EVIDENCE/README.md	./src/models_api/common.pl
./REPOSITORY/EVIDENCE/ocra	./src/patterns.pl
./REPOSITORY/EVIDENCE/ocra/README.md	./src/procs_etb.pl
./REPOSITORY/EVIDENCE/unknown	./src/makefile
./REPOSITORY/EVIDENCE/unknown/README.md	./src/etb_server.pl
./REPOSITORY/EVIDENCE/repository.pl	./src/etb.pl
./REPOSITORY/EVIDENCE/ichecker	./src/agents/ocra_agent.pl
./REPOSITORY/EVIDENCE/ichecker/README.md	./src/agents/axiom_agent.pl
./REPOSITORY/CASES	./src/agents/ichecker_agent.pl
./REPOSITORY/CASES/README.md	./src/agents/certificate_agent.pl
./REPOSITORY/README.md	./src/export.pl
./LICENSE	./src/assurance.pl
./KB/PATTERNS	./src/instantiate.pl
./KB/PATTERNS/patterns_ISO_81001.pl	./src/agent.pl
./KB/PATTERNS/patterns_MILS.pl	./src/command_etb.pl
./KB/MODELS/1.0	./src/demo.pl
./KB/MODELS/1.0/configuration.pl	./src/kb.pl
./KB/MODELS/1.0/system-invariants.inv	./src/com/command.pl
./KB/MODELS/1.0/policy.pl	./src/com/ui.pl
./KB/MODELS/1.0/platform.pl	./src/com/param.pl
./KB/MODELS/1.0/properties	./src/com/test.pl
./KB/MODELS/1.0/properties/p1.pro	./src/com/README.md
./KB/MODELS/1.0/properties/p2.pro	./src/com/procs.pl
./KB/MODELS/1.0/system.bip	./src/evidence.pl
./KB/README.md	
./KB/WORKFLOWS	
./docs/README-O-ETB-v1	
./docs/README-AM-ETB-v3	
./docs/O-ETB-v1.1.0-User-Manual.pdf	
./README.md	
./CAP/README.md	
./BUILD/etb	
./BUILD/README.md	
./TEST/README.md	

8.2 MODULE IDENTIFICATION

The modules contained in the O-ETB v1.1.0 are listed in Table 9 along with, for each module, a brief description, modules it uses, interfaces it provides, and data structures it creates or uses.

Table 9 O-ETB v1.1.0 source code modules

Module	Description	Modules Used	Interfaces Provided	Data Structures
agent	Invokes agent for evidence category; maintains status	evidence	Evidence_validate	Evidence repository
agents	Directory containing separate agent module per evidence category	axiom_agent certificate_agent ichecker_agent ocra_agent	Axiom_validate certificate_validate ichecker_validate ocra_validate	
assurance	Manages AC repository	persistency	Reset_assurance_repository init_assurance_repository attach_assurance_repository insert_ac_instance insert_ac_instance_id_args ac_instance ac_instance_id_args	Persistent data: ac_instance ac_instance_id_args ac_instance_id_counter
command	Command line interpreter (CLI) with command procedures and debug assist	param ui	Enumerated commands	Command syntax, argument specs, help text, operational semantics per command
command_etb	ETB-specific commands; included in command module	etb	Enumerated commands	

Module	Description	Modules Used	Interfaces Provided	Data Structures
etb	Entry point for interactive tool, process arguments, initialize etb, self-test, start CLI	param command test procs models_api patterns assurance evidence agents instantiate export	Command line args Etb etb_server etb_reset	
etb_server	Entry point for server to run in background, initialize, self-test and start server	etb param models_api patterns assurance evidence agents instantiate export etb_server_api	Server invocation	
evidence	Manages evidence repository	persistency	reset_evidence_repository attach_evidence_repository ac_evidence insert_ac_evidence update_ac_evidence update_ongoing	Persistent data: ac_evidence ac_evidence_counter
export	Build and maintain the certification assurance package (CAP) documents	assurance evidence stringutil	Reset_CAP ac_export ac_format	Assurance case and evidence repositories
instantiate	Instantiate a parameterized AC pattern using actual parameters	models_api patterns assurance evidence agent	instantiate_pattern instantiate_pattern_list	Dynamic data: Ac_pattern_pending Ac_pattern_running
kb	Knowledge base operations (placeholder)	patterns assurance evidence	load_specification_from_file kb_reset	Knowledge base: models, patterns, workflows, …

Module	Description	Modules Used	Interfaces Provided	Data Structures
models_api	Directory containing modules that provide access to models in the knowledge base	common configuration model platform policy	Interfaces provided by the used modules	Static data: models in the MODELS section of the Knowledge Base (KB)
param	Global parameter file; file and directory names, switches, settings, etc	N/A	Various parameter values and interfaces to construct complex parameters from primitive parameter values	Self-contained data
patterns	Provides interface to multiple AC pattern files	Pattern files specified as param:pattern_files	Pattern library init ac_pattern	Patterns contained in the various pattern files
procs	Stored CLI procedures	CLI commands	General defined procedures by name	Self-contained proc definitions
procs_etb	ETB-specific CLI procs	CLI commands	ETB-specific defined procedures by name	Self-contained proc definitions
stringutil	Utility functions	N/A	Special-purpose string manipulation operations	N/A
test	Utility functions	Self-test modules declared in param	Operations to invoke individual tests or predefined sets of tests	Test cases defined in individual self-test modules
ui	Utility functions	N/A	Basic terminal interaction ops	N/A

9. ANNEX B: POPULATING THE O-ETB KNOWLEDGE BASE

9.1 MODEL INFORMATION

9.1.1 Architectural model

A key principle [Kelly] applied in O-ETB is that of structuring the assurance case according to the architecture of the system to be assured. This is reflected in the argument pattern language as constructs to iterate over architectural entities. The model-related terminology currently utilized in O-ETB is based on an architectural meta-model developed in connected with the MILS architectural approach [Boettcher, et al] as manifest in the EC's D-MILS and CITADEL projects.

The meta-model's terminology includes some distinct terms of art such as the "planes" of a system: "foundational plane", "operational plane", "configuration plane", "monitoring plane", "adaptation plane" and "certification assurance plane". Despite the distinctness of these terms, familiar terms and architectural constructs can be mapped to the meta-model relatively easily. However, since the meta-model is intended to encompass distributed, dynamically reconfigurable and adaptive systems that are capable of just-in-time runtime certification, not all of the planes of the meta-model are embodied in non-distributed or non-adaptive concrete systems. Furthermore, the certification assurance plane is a design and implementation time activity and not a runtime service for systems that do not require runtime recertification after adaptation.

Of ubiquitous interest is the foundational plane, more commonly thought of as the "platform". It includes the computer processor(s), memories, storage devices, network devices, sensors and actuators, their device drivers and the operating system(s) running on the processors. In some cases the platform includes higher-level middleware that provides communication and coordination services. If the platform is distributed then each participating computer is referred to as a "node" connected to other nodes by the network. Nodes can be autonomous or coordinated by the platform software.

The applications, possibly multiple interacting applications, reside in the operational plane (or planes). The term "application space" might be more natural. Thus we have adapted the meta-model terms "foundational plane" = "platform" and "operational plane" = "application space" for our present purposes.

Both the platform and the application space may each be represented as composed of distinct but interacting components, commonly represented in component diagrams. The components of the hardware platform benefit from physical isolation and physical/electrical interfaces, and in turn can execute software components and provide logical isolation and discrete communication as a service to other software component architectures. The software platform components benefit from logical isolation and controlled interactions, via typed

ports, enforced by the hardware/OS platform. In this way properties of components and contracts among components can be specified and used for verification of compositions.

The model information as needed by the O-ETB is expressed in the terms of the meta-model described above but the translation can be made from other convenient and more familiar terms on import of the model information. The needed architectural model information be retrieved from the MedSecurance modelling tools in a mutually agreed common exchange format which can then be subject to a straightforward translation and import into the KB as facts in the terminology and representation of the meta-model.

Informally, the terms of the meta-model are (we further use the term "policy architecture" or PA to signify an instance of the meta-model):

- Nodes (of a PA)
- Modes (of a Subject)
- Subjects (of a PA)
- Subject-Subject Flows (of a PA)
- Transitions (of a Subject)

Each of these may be iterated over in an AC pattern. Further, there can be an iteration over a specified list of constant values. Please refer to the expanded description of the pattern instantiation procedure in Section 3.3.2.3 of D3.1.

9.1.2 Properties

Properties are expressed in Linear Temporal Logic (LTL).

9.2 ARGUMENT PATTERNS

Argument patterns are expressed in the notation introduced in Section 2.3 of this document and in Section 3.3.2.3 of D3.1.

9.3 WORKFLOW INFORMATION

9.3.1 Tool flows

Tool flows are currently determined by the defined assurance case argument patterns and the semantics of the instantiation procedure.

9.3.2 Workflow programs

This refers to a capability resulting from the anticipated future inclusion of a version of Workflow Prolog in the O-ETB implementation.

9.4 CERTIFICATION REQUIREMENTS

9.4.1 Content/process requirements

A certification regime, a regulation, or a standard may levy prescriptive requirements for specific assurance activities and/or specific content to be presented as part of the technical data (resulting from prescribed processes) submitted in support of an application for certification.

9.4.2 Goal-based requirements

A certification regime, a regulation, or a standard may establish goals that must be achieved by the certification applicant rather than prescribed actions that must be shown to have been done.

9.4.3 Presentation requirements

A certification regime may establish a preferred or required form for the presentation of evidence in support of a certification application.

9.5 EVIDENCE-RELATED KNOWLEDGE

9.5.1 Evidence categories and artefact kinds

An ***evidence category*** associates evidence in a specific form with one or more activities or tools that generate evidence artefacts of that category.

An ***artefact kind*** is a collection of one or more evidence categories that are used in a similar role in an overall assurance strategy.

Table 10 Evidence artefacts and categories

Artefact kind			
Evidence Category	***Description***	***Form***	***Activity/Tool***
Specification artefacts			
Architecture specification		Id of system components and connections by ports	
Invariant specification		State formula	
Property specification		LTL formula	
Contract specification	A relationship among components, pairwise or in	Component assume-	OCRA

Artefact kind			
Evidence Category	***Description***	***Form***	***Activity/Tool***
	groups, that defines interdependencies	guarantee spec	
Verification artefacts			
Invariant verification			
Property verification			
Safety analysis artefacts			
Fault tree			
FMEA table			
Validation artefacts			
Contract consistency			
Contract refinement			
Configuration artefacts			
Resource allocation		Configuration file <format>	
Performance artefacts			
Performance analysis			
Standards conformance artifacts			
Compliance sheet			
Conformance certificate			

9.5.2 Evidential tools

9.6 GENERAL KNOWLEDGE

Facts and rules encoded by experts to be used to refine operation.

10. ANNEX C: O-ETB TESTING

Some groundwork for O-ETB testing has been laid in the current version.

10.1 FUNCTIONAL TESTS

The functional testing of the Open Evidential Tool Bus (O-ETB), is accomplished with a combination of two mechanisms:

- a built-in self-test feature, accessible with the `selftest` command of the 'etb' Assurance Tool, that exercises key functions through internal interfaces, and,
- shell scripts that exercise the RESTful APIs to the same functions.

Both of these methods are used as regression tests during ongoing development and maintenance of O-ETB.

Beyond functional correctness, tests will be developed for volume and load, performance and endurance, and correctness tests for non-functional support as described below.

10.2 VOLUME AND LOAD TESTS

For volume and load tests a simple framework of scripts was constructed. The top level script `stress-top` first invokes a `stress-etbapi` script that calls the Assurance Server's interface to establish a session to be used for subsequent assurance case and evidence API tests and then initiates multiple parallel copies of a script that repeatedly makes a series of API calls. The queries made by this script comprise ….
The `stress-top` script takes a numeric argument that it passes along to each of the sub-scripts to tell them how many iterations of the test cases to perform. The value used for this count should be sufficient to assure that the several sub-scripts are running in parallel. The sub-script checks the results of each query, according to the expected result. Any failure of these comparisons is reported as an "ERROR" output along with the failing test case identifier.

10.3 PERFORMANCE AND ENDURANCE TESTS

To facilitate performance tests, a `time` command is provided by the 'etb' Policy Tool to allow the user to time the execution of any single 'etb' command, or to time multiple executions of a single command or a command script. In the case that multiple executions are requested, text output from the repeated command executions are sent to a null stream, so as not to overly bias the timing result for the execution of the core functions by the accumulated latency of file or terminal I/O resulting from working through the interactive command interpreter.

This `time` command is used to measure the time to perform multiple executions of a representative Assurance Tool command:

```
instantiate( … ).
```

This query is by far the most frequently occurring work of the O-ETB system, others being one-time or infrequent calls, which are not only less frequent but much less computationally demanding. Consequently, this test approach is deemed to provide adequate practical coverage.

correctness of the AC instantiation function. Therefore, a series of tests has been developed to demonstrate this correctness, at the command level, at the HTTP API level, and as self-tests perfomed at startup or on demand. The self-tests are constructed within a test framework organized by source module that facilitates addition of tests for new modules and extension of existing module tests. The correctness test framework compares execution results to known answers for the test cases and report discrepancies.
The 'etb' Assurance Tool has built-in self-tests. These may be run to ensure that expected results are produced after O-ETB installation, or as a regression test during further code development. The self-tests can be run by starting 'etb' normally and entering at the "O-ETB>" prompt the command "selftest" or "regtest", which run all of the currently configured tests.
Self-tests and regression tests use the test framework implemented by the module `test.pl`. Tests for any module may be created in a similar fashion to the currently implemented tests, by creating a `TEST/mod_test.pl` file for a corresponding module file `mod.pl`, which should have an include directive for its corresponding test file. Tests defined as startup tests are run by the selftest command and those defined as regression tests will be run in addition to the defined self-tests by the regtest command.
The file TEST/instantiate_test.pl defines self-test cases with expected results for the pattern instantiation functions in the code file `instantiate.pl`. A representative sample of currently configured test cases are shown in Table 11.

Table 11 Pattern instantiation correctness tests

Test case ID	Test case	Expected result
tc01		Succeed
tc02		Succeed
tc03		Succeed
tc04		Fail
tc05		Fail

10.4 HTTP API CORRECTNESS TESTS

The HTTP APIs are tested using an environment-provided utility command (`curl`) that enables HTTP requests to be constructed and sent to a network port on the local or a remote machine (the current tests use a fixed reserved port on the local host). The tests are packaged as shell script files in the TEST/nn-tests directory, with each file name beginning with a number. The directory also contains for each such script a file of the expected result

in JSON format. A master script, `run-nn-tests.sh`, runs each of the individual numbered test scripts in turn and checks the current results against expected results. Table 12 identifies the automated HTTP API tests currently defined. Every such test is represented by a shell script file with the name corresponding to the Test case ID, and an expected result files (e.g. 01-expected-json) and therefore, as the expected results can be rather lengthy, the expected result is not given here in Table 12.

Table 12 Assurance Server HTTP API correctness tests

Test case ID	Test case description